

ECE 315

Steven Knudsen, PhD, PEng

Lecture 22 :

- Memory mapping to peripherals



UNIVERSITY
OF ALBERTA

Last Lecture

- RTM (refer to the manual)
- Memory and memory-mapped I/O

This Lecture

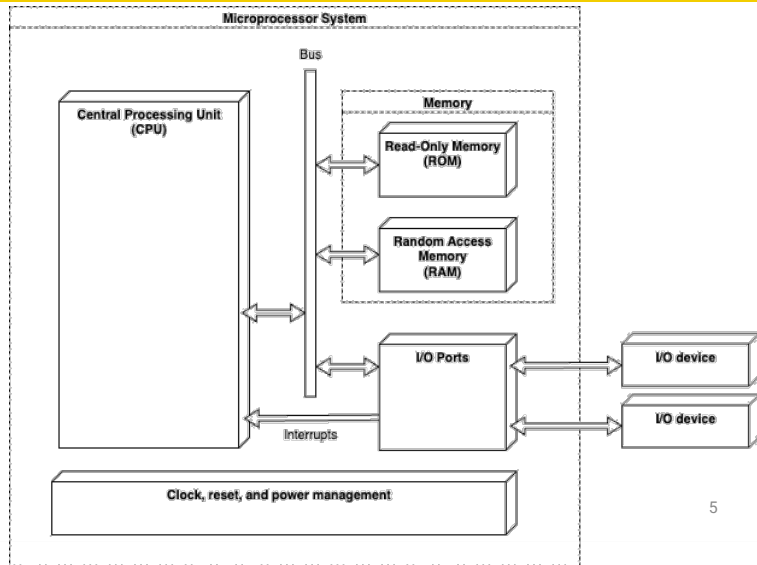
- Memory mapping to peripherals

Learning Objectives

- Explain the concept of memory-mapped I/O
- Describe the logical architecture of a typical microprocessor in the context of memory-mapped peripherals
- Understand the RP2040 memory map as an example
- From the example memory-mapped peripheral C program, analyze how SPI communication is implemented using direct register manipulation

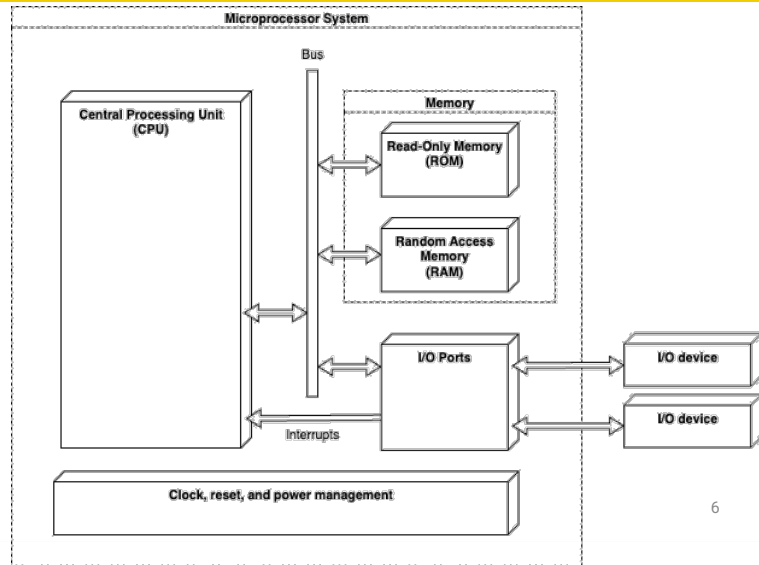
Logical Architecture

- Typical microprocessor
- The I/O ports can support serial or parallel interfaces to external devices
- Internal devices may be treated as addressable memory,



Logical Architecture

- Typical microprocessor
- The I/O ports can support serial or parallel interfaces to external devices
- Internal devices may be treated as addressable memory, aka
- **Memory-mapped peripherals**



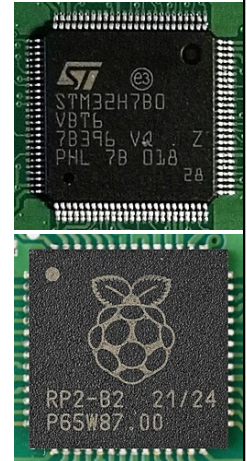
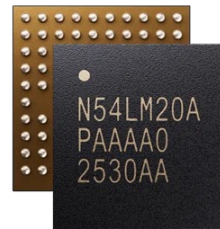
Advanced RISC Machines – ARM

- A very popular, common computer architecture was developed by companies that eventually became ARM starting in the early 80s
- It's not perfectly correct to say "computer architecture" because ARM licenses a family of Reduced Instruction Set Computer (RISC) instruction set architectures
 - ARM specifies the instructions and companies build hardware to execute them
- Over 200 billion ARM-based computers sold so far (at least!)

We are going to use an ARM processor to illustrate memory-mapped peripherals

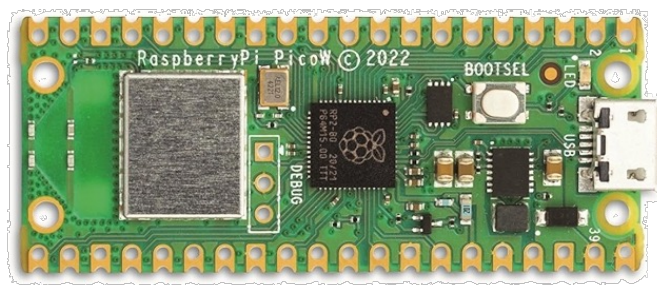
Examples

- We look to a few example that are readily available
- STM32 - a family of 32-bit microcontroller and microprocessor integrated circuits by STMicroelectronics.
- Pico (RP2040, RP2350) –a 32-bit dual-core ARM Cortex-M0+ microcontroller designed by Raspberry Pi Ltd.
- nRF54LM20A – 32-bit ARM Cortex-M33 microcontroller from Nordic Semi.
 - Excellent embedded Bluetooth processor(!)



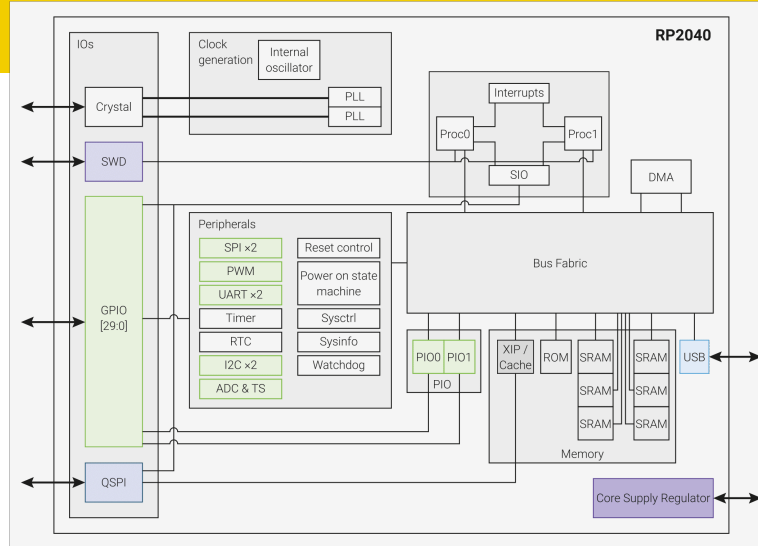
RP2040 and Cortex-M0+

- Use ARM to illustrate memory-mapped peripherals
- Focus on the RP2040 since it is the processor on the Raspberry Pi Pico
 - The Pico is used in multiple MCTR courses
- The ARM architecture it's based on is the Cortex-M0+



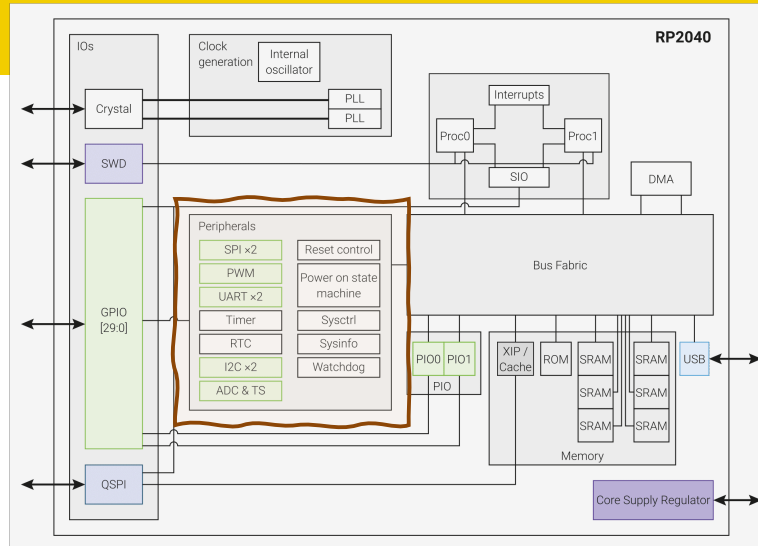
RP2040 Architecture

- Many peripherals



RP2040 Architecture

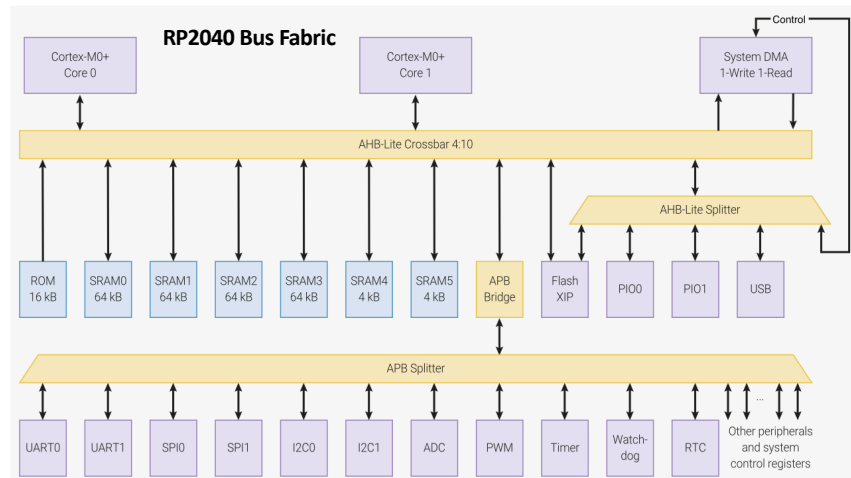
- Many peripherals
- Dual processors, Proc0 and Proc1
- SPI, I2C, UART, plus
- Timer
- Real-time Clock (RTC)
- Analogue-to-Digital Convertor (ADC)



RP2040 Architecture – more detail, bus fabric

AHB is the **Advanced High-performance Bus** matrix (crossbar)

- AHB bus decoders match the address to the peripheral
- AHB-Lite system typically has:
 - One AHB master (the Cortex-M0/M0+)
 - Multiple AHB slaves (Flash, SRAM, DMA, AHB-to-APB bridge, etc.)
 - A bus decoder to select which slave responds
- APB = **Advanced Peripheral Bus** maps peripherals in memory space



Main reference for the RP2040 material is the RP2040 Datasheet, 2025-02-20

12

RP2040 Memory Map

Area Name	Description	Starting Address
ROM	Read-Only Memory	0x00000000
XIP	eXecute-In-Place flash memory	0x10000000
SRAM	Static Random-Access Memory	0x20000000
APB Peripherals	A dvanced P eripheral B us mapped peripherals	0x40000000
AHB-Lite Peripherals	A dvanced H igh-Performance B us mapped peripherals	0x50000000
IOPORT Registers	I nput- O utput P ort registers	0xD0000000
Cortex-M0+ internal registers	Various registers	0xE0000000

RP2040 Memory Map

Area Name	Description	Starting Address
ROM	Read-Only Memory	0x00000000
XIP	eXecute-In-Place flash memory	0x10000000
SRAM	Static Random-Access Memory	0x20000000
APB Peripherals	A dvanced P eripheral B us mapped peripherals	0x40000000
AHB-Lite Peripherals	A dvanced H igh-Performance B us mapped peripherals	0x50000000
IOPORT Registers	I nput- O utput P ort registers	0xD0000000
Cortex-M0+ internal registers	Various registers	0xE0000000

RP2040 Memory Map

- From the Cortex-M0+ Devices Generic User Guide
- A more accurate view of how memory is organized in terms of address space
- This is a picture of what can be supported, not what necessarily is
- The Pico supports
 - 264kB on-chip (RP2040) SRAM, not the whole 512MB SRAM possible
 - 2MB on-board FLASH (code) memory, not the whole 512MB Code storage possible
 - APB and AHB-Lite Peripherals

System	511MB	0xFFFFFFFF
Private Peripheral Bus	1MB	0xE0100000 0xE00FFFFF 0xE0000000 0xDFFFFFFF
External device	1.0GB	
External RAM	1.0GB	0xA0000000 0x9FFFFFFF
Peripheral	0.5GB	0x60000000 0x5FFFFFFF
SRAM	0.5GB	0x40000000 0x3FFFFFFF
Code	0.5GB	0x20000000 0x1FFFFFFF
		0x00000000

RP2040 Memory Map

- From the Cortex-M0+ Devices Generic User Guide
- A more accurate view of how memory is organized in terms of address space
- This is a picture of what can be supported, not what necessarily is
- The Pico supports
 - 264kB on-chip (RP2040) SRAM, not the whole 512MB SRAM possible
 - 2MB on-board FLASH (code) memory, not the whole 512MB Code storage possible
 - **APB and AHB-Lite Peripherals**

System	511MB	0xFFFFFFFF
Private Peripheral Bus	1MB	0xE0100000 0xE00FFFFF 0xE0000000 0xDFFFFFFF
External device	1.0GB	
External RAM	1.0GB	0xA0000000 0x9FFFFFFF
Peripheral	0.5GB	0x60000000 0x5FFFFFFF 0x40000000 0x3FFFFFFF
SRAM	0.5GB	
Code	0.5GB	0x20000000 0x1FFFFFFF 0x00000000

Mapped APB Peripherals

- Some example peripherals
 - CLOCKS – provides independent clocks to on-chip and external components
 - RESETS – supports software control of peripherals not critical to boot the processor, e.g., the USB controller
 - IO_BANK0 – the base interface to the RP2040's 36 multi-functional General-Purpose Input/Output (GPIO) pins
 - UART0 – access to the one of two universal asynchronous receiver-transmitter interfaces
 - ADC – a 12-bit analogue-to-digital converter with five multiplexed inputs



SYSINFO_BASE	0x40000000
SYSCFG_BASE	0x40004000
CLOCKS_BASE	0x40008000
RESETS_BASE	0x4000c000
PSM_BASE	0x40010000
IO_BANK0_BASE	0x40014000
IO_QSPI_BASE	0x40018000
PADS_BANK0_BASE	0x4001c000
PADS_QSPI_BASE	0x40020000
XOSC_BASE	0x40024000
PLL_SYS_BASE	0x40028000
PLL_USB_BASE	0x4002c000
BUSCTRL_BASE	0x40030000
UART0_BASE	0x40034000
UART1_BASE	0x40038000
SPI0_BASE	0x4003c000
SPI1_BASE	0x40040000
I2C0_BASE	0x40044000
I2C1_BASE	0x40048000
ADC_BASE	0x4004c000
PWM_BASE	0x40050000
TIMER_BASE	0x40054000
WATCHDOG_BASE	0x40058000
RTC_BASE	0x4005c000
ROSC_BASE	0x40060000
VREG_AND_CHIP_RESET_BASE	0x40064000
TBMAN_BASE	0x4006c000

RP2040 Memory Map – AHB-Lite Peripherals

- The Advanced High-Performance Bus is based on a Direct Memory Access (DMA) controller
- It coordinates and performs bulk data transfers on the processor's behalf, leaving it to do other things
- The throughput using the DMA is much faster than having the processor move data
- At base address of `0x50000000` various DMA registers are mapped
 - The programmer writes or reads data at offsets corresponding to specific functional registers

Memory-Mapped Peripheral Example

- We'll get to DMA next lecture, but first an example
- Consider using the RP2040 SPI to talk to an accelerometer
- Will look at only part of a C program that uses memory-mapped registers to transfer data via SPI

SPI->Accel

Define the memory-mapped locations for the SPI, resets, I/O locations

```
1  /*
2  * Cortex-M0+ SPI Memory-Mapped Communication Example
3  * Target: RP2040 (Raspberry Pi Pico)
4  * Peripheral: ADXL345 Accelerometer
5  *
6  * This example demonstrates:
7  * - Memory-mapped SPI register configuration
8  * - SPI communication protocol (read/write)
9  * - GPIO configuration for SPI pins and chip select
10 * - Reading acceleration data from a real peripheral
11 */
12
13 #include <stdint.h>
14
15 // =====
16 // RP2040 Memory-Mapped Register Definitions
17 // =====
18
19 // Base addresses
20 #define SPI0_BASE      0x4003C000
21 #define SPI1_BASE      0x40040000
22 #define RESETS_BASE    0x4000C000
23 #define IO_BANK0_BASE  0x40014000
24 #define PADS_BANK0_BASE 0x4001C000
25 #define SIO_BASE       0xD0000000
```

SPI->Accel

More definitions, some of which make use of the memory-mapped locations

```
34 // SPI Status Register bits
35 #define SPI_SR_TFE      (1 << 0) // Transmit FIFO empty
36 #define SPI_SR_TNF      (1 << 1) // Transmit FIFO not full
37 #define SPI_SR_RNE      (1 << 2) // Receive FIFO not empty
38 #define SPI_SR_RFF      (1 << 3) // Receive FIFO full
39 #define SPI_SR_BSY      (1 << 4) // SPI busy flag
40
41 // Reset Control
42 #define RESETS_RESET     (*((volatile uint32_t*)(RESETS_BASE + 0x00)))
43 #define RESETS_RESET_DONE (*((volatile uint32_t*)(RESETS_BASE + 0x08)))
44 #define RESET_SPI0      (1 << 16)
45
46 // GPIO Function Select (for SPI pins)
47 #define GPIO_FUNC_SPI    1
48 #define GPIO_FUNC_SIO    5
49
50 // GPIO Registers (SIO - Single-cycle IO)
51 #define SIO_GPIO_OUT      (*((volatile uint32_t*)(SIO_BASE + 0x010)))
52 #define SIO_GPIO_OUT_SET  (*((volatile uint32_t*)(SIO_BASE + 0x014)))
53 #define SIO_GPIO_OUT_CLR  (*((volatile uint32_t*)(SIO_BASE + 0x018)))
54 #define SIO_GPIO_OE       (*((volatile uint32_t*)(SIO_BASE + 0x020)))
55 #define SIO_GPIO_OE_SET   (*((volatile uint32_t*)(SIO_BASE + 0x024)))
56 #define SIO_GPIO_OE_CLR   (*((volatile uint32_t*)(SIO_BASE + 0x028)))
```



21

SPI->Accel

Accelerometer register address definitions.

It, too, has a memory-mapping, but one independent of the Cortex-M0+ (RP2040)



22

```
58 // =====
59 // ADXL345 Accelerometer Register Definitions
60 // =====
61
62 #define ADXL345_DEVID      0x00 // Device ID (should read 0xE5)
63 #define ADXL345_POWER_CTL  0x20 // Power control
64 #define ADXL345_DATA_FORMAT 0x31 // Data format control
65 #define ADXL345_DATAX0     0x32 // X-axis data LSB
66 #define ADXL345_DATAX1     0x33 // X-axis data MSB
67 #define ADXL345_DATAY0     0x34 // Y-axis data LSB
68 #define ADXL345_DATAY1     0x35 // Y-axis data MSB
69 #define ADXL345_DATAZ0     0x36 // Z-axis data LSB
70 #define ADXL345_DATAZ1     0x37 // Z-axis data MSB
71
72 // ADXL345 Commands
73 #define ADXL345_READ_BIT    (1 << 7) // Set for read operations
74 #define ADXL345_MB_BIT     (1 << 6) // Multi-byte read bit
75
76 // Power Control bits
77 #define ADXL345_MEASURE     (1 << 3) // Enable measurement mode
78
79 // =====
80 // Pin Definitions for SPI0
81 // =====
82
83 #define PIN_MISO 16 // SPI0 RX
84 #define PIN_CS   17 // Chip Select (manual GPIO)
85 #define PIN_SCK  18 // SPI0 SCK
86 #define PIN_MOSI 19 // SPI0 TX
87
```

SPI->Accel – Main program; initializes SPI

```
114 int main(void) {
115     int16_t accel_x, accel_y, accel_z;
116     uint8_t device_id;
117
118     // Initialize SPI and GPIO
119     spi_init();
120     gpio_init_spi();
121
122     // Small delay for power stabilization
123     delay_ms(100);
124
125     // Verify device ID
126     device_id = spi_read_byte(ADXL345_DEVID);
127
128     // device_id should be 0xE5 for ADXL345
129     if (device_id != 0xE5) {
130         // Device not responding or wrong device
131         while(1) {
132             // Error condition - could toggle LED here
133         }
134     }
135
136     // Initialize the accelerometer
137     adxl345_init();
138
139     // Main loop - continuously read acceleration data
140     while(1) {
141         adxl345_read_accel(&accel_x, &accel_y, &accel_z);
142
143         // Process the data here
144         // accel_x, accel_y, accel_z contain raw acceleration values
145         // Scale: ~256 LSB/g in default ±2g range
146
147         delay_ms(100); // 10 Hz update rate
148     }
149
150     return 0;
151 }
```

In the main program, the SPI initialization function is of interest, it's where the memory-mapped registers for SPI0 will be accessed

SPI->Accel

spi_init() makes use of the memory-mapped locations



24

```
157 void spi_init(void) {
158     // Release SPI0 from reset
159     RESETS_RESET &= ~RESET_SPI0;
160
161     // Wait for reset to complete
162     while (!(RESETS_RESET_DONE & RESET_SPI0));
163
164     // Disable SPI during configuration
165     SPI0_SSPCR1 = 0;
166
167     // Configure SPI0 Control Register 0
168     // DSS = 0b0111 (8-bit data)
169     // FRF = 0b00 (Motorola SPI frame format)
170     // SP0 = 1 (Clock polarity: idle high for ADXL345)
171     // SPH = 1 (Clock phase: capture on second edge)
172     // SCR = 0 (Serial clock rate divider)
173     SPI0_SSPCR0 = (7 << 0) | // 8-bit data
174                   (0 << 4) | // Motorola format
175                   (1 << 6) | // SP0 = 1
176                   (1 << 7) | // SPH = 1
177                   (0 << 8); // SCR = 0
178
179     // Set clock prescale divider
180     // System clock is 125 MHz, we want ~1 MHz SPI clock
181     // Clock = sys_clk / (CPSDVSR * (1 + SCR))
182     // 125 MHz / 125 = 1 MHz
183     SPI0_SSPCPSR = 125;
184
185     // Enable SPI, set as master
186     SPI0_SSPCR1 = (1 << 1); // SSE = 1 (enable), MS = 0 (master)
187 }
```


SPI->Accel

spi_init() makes use of the memory-mapped locations

SPI0_SSPCR1=0 clears that location, disables SPI0

SPI0_SSPCR1=125 sets that location to 0x7D to configure the SPI mode

Finally,

SPI0_SSPCR1= (1<<1) enables SPI0



25

```

25 #define SIO_BASE      0x00000000
26
27 // SPI Registers (using SPI0)
28 #define SPI0_SSPCR0    (*(volatile uint32_t*)(SPI0_BASE + 0x00)) // Control register 0
29 #define SPI0_SSPCR1    (*(volatile uint32_t*)(SPI0_BASE + 0x04)) // Control register 1
30 #define SPI0_SSPDR     (*(volatile uint32_t*)(SPI0_BASE + 0x08)) // Data register
31 #define SPI0_SSPCR     (*(volatile uint32_t*)(SPI0_BASE + 0x0C)) // Status register
32 #define SPI0_SSPCPSR   (*(volatile uint32_t*)(SPI0_BASE + 0x10)) // Clock prescale
33
165 SPI0_SSPCR1 = 0;
166
167 // Configure SPI0 Control Register 0
168 // DSS = 0b0111 (8-bit data)
169 // FRF = 0b00 (Motorola SPI frame format)
170 // SP0 = 1 (Clock polarity: idle high for ADXL345)
171 // SPH = 1 (Clock phase: capture on second edge)
172 // SCR = 0 (Serial clock rate divider)
173 SPI0_SSPCR0 = (7 << 0) | // 8-bit data
174                 (0 << 4) | // Motorola format
175                 (1 << 6) | // SP0 = 1
176                 (1 << 7) | // SPH = 1
177                 (0 << 8); // SCR = 0
178
179 // Set clock prescale divider
180 // System clock is 125 MHz, we want ~1 MHz SPI clock
181 // Clock = sys_clk / (CPSDVSR * (1 + SCR))
182 // 125 MHz / 125 = 1 MHz
183 SPI0_SSPCPSR = 125;
184
185 // Enable SPI, set as master
186 SPI0_SSPCR1 = (1 << 1); // SSE = 1 (enable), MS = 0 (master)
187 }

```

```

225 // =====
226 // SPI Transfer Function
227 // =====
228
229 uint8_t spi_transfer(uint8_t data) {
230     // Wait for TX FIFO to have space
231     while (!(SPI0_SSPSR & SPI_SR_TNF));
232
233     // Write data to transmit
234     SPI0_SSPCR = data;
235
236     // Wait for RX FIFO to have data
237     while (!(SPI0_SSPSR & SPI_SR_RNE));
238
239     // Read and return received data
240     return (uint8_t)SPI0_SSPCR;
241 }

```

The transfer function only refers to memory-mapped locations to manage the transfer of the data

`spi_read_byte` manages the chip select signal and uses `spi_transfer` to do the work

```

259 uint8_t spi_read_byte(uint8_t reg) {
260     uint8_t result;
261
262     CS_LOW();
263
264     // Send register address with read bit set
265     spi_transfer(reg | ADXL345_READ_BIT);
266
267     // Read data
268     result = spi_transfer(0x00); // Clock out dummy byte to read
269
270     CS_HIGH();
271
272     return result;
273 }

```

Summary

- Microprocessors use memory-mapped I/O to access peripherals via normal memory addresses
 - For example, for the Cortex-M0+ (RP2040) peripherals accessed through APB & AHB buses
 - RP2040 memory map includes ROM, Flash (XIP), SRAM, APB peripherals, AHB-Lite peripherals, and GPIO registers
 - Examples of mapped peripherals: CLOCKS, RESETS, GPIO, UART, ADC, TIMER, PWM, DMA
- SPI example shows direct register manipulation for peripheral configuration and data transfer

Next Lecture

- Direct Memory Access (DMA)
- DMA Example (RP2040)

Questions



<https://xkcd.com/768/>

Questions

